

UNIDAD 1

ARQUITECTURAS WEB

1. CARACTERÍSTICAS DE LA PROGRAMACIÓN WEB.....	1
1.1. PÁGINAS WEB ESTÁTICAS Y DINÁMICAS.....	2
VENTAJAS Y LIMITACIONES DE LAS PÁGINAS ESTÁTICAS.....	4
1.2. APLICACIONES WEB.....	5
1.2. EJECUCIÓN DE CÓDIGO EN EL SERVIDOR Y EN EL CLIENTE.....	6
2. TECNOLOGÍAS PARA PROGRAMACIÓN WEB DEL LADO DEL SERVIDOR.....	8
2.1. ARQUITECTURAS Y PLATAFORMAS.....	10
2.1.1. SELECCIÓN DE UNA ARQUITECTURA DE PROGRAMACIÓN WEB.....	12
2.2. INTEGRACIÓN CON EL SERVIDOR WEB.....	13
3. LENGUAJES DE SERVIDOR.....	14
3.1. CÓDIGO EMBEBIDO EN EL LENGUAJE DE MARCAS.....	15
3.2. HERRAMIENTAS DE PROGRAMACIÓN.....	15
3.3. PROGRAMACIÓN WEB CON PHP.....	16
4. PUESTA EN MARCHA.....	17

1. CARACTERÍSTICAS DE LA PROGRAMACIÓN WEB.

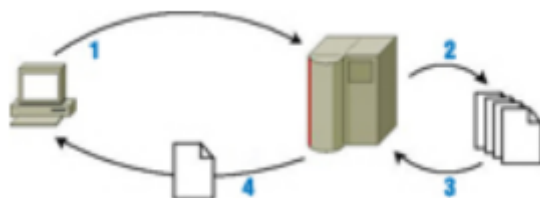
Seguro que **ya sabes** exactamente qué es una **página web**, e incluso conozcas cuáles son los pasos que se suceden para que, cuando visitas una web poniendo su dirección en el navegador, la página se descargue a tu equipo y se pueda mostrar. Sin embargo, este procedimiento que puede parecer sencillo, a veces no lo es tanto. Todo depende de cómo se haya hecho esa página.

Cuando una **página web se descarga** en tu equipo, su contenido define qué se debe mostrar en pantalla. Este contenido está programado en un **lenguaje de marcado**, formado por etiquetas, que puede ser **HTML** o **XHTML**. Las etiquetas que componen la página indican el objetivo de cada una de las partes que la componen. Así, dentro de estos lenguajes hay etiquetas para indicar que un texto es un encabezado, que forma parte de una tabla, o que simplemente es un párrafo de texto.

Además, si la página está bien estructurada, la información que le indica al navegador el **estilo** con que se debe mostrar cada parte de la página estará almacenado en otro fichero, una **hoja de estilos o CSS**. La hoja de estilos se encuentra indicada en la página web y el navegador la descarga junto a ésta. En ella nos podemos encontrar, por

ejemplo, estilos que indican que el encabezado debe ir con tipo de letra Arial y en color rojo, o que los párrafos deben ir alineados a la izquierda.

Estos dos ficheros se descargan a tu ordenador desde un servidor web como respuesta a una petición. El proceso es el que se refleja en la siguiente figura.



Los pasos son los siguientes:

1. **Tu ordenador solicita a un servidor web una página** con extensión .htm, .html o .xhtml.
2. **El servidor busca esa página** en un almacén de páginas (cada una suele ser un fichero).
3. Si **el servidor encuentra esa página**, la recupera.
4. Y por último **la envía al navegador** para que éste pueda mostrar su contenido.

Este es un ejemplo típico de una **comunicación cliente-servidor**. El cliente es el que hace la petición e inicia la comunicación, y el servidor es el que recibe la petición y la atiende. El navegador es el cliente web.

1.1. PÁGINAS WEB ESTÁTICAS Y DINÁMICAS.

Las páginas que viste en el ejemplo anterior (HTML + CSS) se llaman **páginas web estáticas**. Estas páginas se encuentran almacenadas en su forma definitiva, tal y como se crearon, y su contenido no varía. Son útiles para mostrar una información concreta, y mostrarán esa misma información cada vez que se carguen. La única forma en que pueden cambiar es si un programador la modifica y actualiza su contenido.

En contraposición a las páginas web estáticas, como ya te imaginarás, existen las **páginas web dinámicas**. Estas páginas, como su nombre indica, se caracterizan porque su contenido cambia en función de diversas variables, como puede ser el navegador que estás usando, el usuario con el que te has identificado, o las acciones que has efectuado con anterioridad.

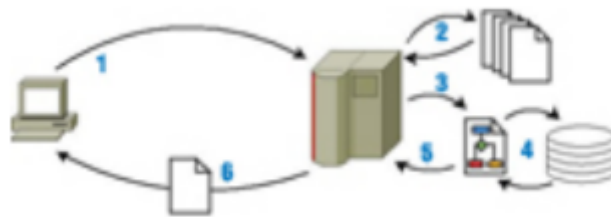
Dentro de las **páginas web dinámicas**, es muy importante distinguir **dos tipos**:

- Aquellas que **incluyen código que ejecuta el navegador**. En estas páginas el código ejecutable, normalmente en **lenguaje JavaScript**, se incluye dentro del

HTML (o XHTML) y se descarga junto con la página. Cuando el navegador muestra la página en pantalla, ejecuta el código que la acompaña. Este código puede incorporar múltiples funcionalidades que pueden ir desde mostrar animaciones hasta cambiar totalmente la apariencia y el contenido de la página. En este módulo no vamos a ver JavaScript, salvo cuando éste se relaciona con la programación web del lado del servidor.

- El segundo tipo son aquellas páginas cuyo código se ejecuta en el servidor antes de enviar el resultado al navegador. Como ya sabes, hay muchas páginas en Internet que no tienen extensión .htm, .html o .xhtml. Muchas de estas páginas tienen extensiones como .php, .asp, .jsp, .cgi o .aspx. En éstas, el contenido que se descarga al navegador es similar al de una página web estática: HTML (o XHTML). Lo que cambia es la forma en que se obtiene ese contenido. Al contrario de lo que vimos hasta ahora, esas páginas no están almacenadas en el servidor; más concretamente, el contenido que se almacena no es el mismo que después se envía al navegador. **El HTML de estas páginas se forma como resultado de la ejecución de un programa**, y esa ejecución tiene lugar en el servidor web (aunque no necesariamente por ese mismo servidor).

El esquema de funcionamiento de una página web dinámica es el siguiente:



1. **El cliente web** (navegador) de tu ordenador **solicita** a un servidor web una página web.
2. El **servidor busca** esa página y la recupera.
3. En el caso de que se trate de una **página web dinámica**, es decir, que su contenido deba ejecutarse para obtener el HTML que se devolverá, el **servidor web** contacta con el módulo responsable de **ejecutar el código** y se lo envía.
4. Como parte del **proceso de ejecución**, puede ser necesario obtener información de algún **repositorio**, como por ejemplo consultar registros almacenados en una base de datos.
5. El **resultado** de la ejecución será una página en **formato HTML**, similar a cualquier otra página estática.
6. El **servidor web envía el resultado** obtenido al navegador, que la procesa y muestra en pantalla.

Este procedimiento tiene lugar constantemente mientras consultamos páginas web. Por ejemplo, cuando consultas tu correo en Gmail, Hotmail, Yahoo o cualquier otro servicio de correo vía web, lo primero que tienes que hacer es introducir tu nombre de usuario y contraseña. A continuación, lo más habitual es que el servidor te muestre una

pantalla con la bandeja de entrada, en la que aparecen los mensajes recibidos en tu cuenta. Esta pantalla es un claro ejemplo de una página web dinámica.

Obviamente, el servidor no envía esa misma página a todos los usuarios, sino que la **genera de forma dinámica** en función de quién sea el usuario que se conecte. Para generarla, el servidor ejecuta un programa que obtiene los datos de tu usuario (tus contactos, la lista de mensajes recibidos) y con ellos compone la página web que recibes desde el servidor web.

VENTAJAS Y LIMITACIONES DE LAS PÁGINAS ESTÁTICAS

Aunque la utilización de páginas web dinámicas te parezca la mejor opción para construir un sitio web, no siempre lo es. Sin lugar a dudas, es la que más potencia y flexibilidad permite, pero las páginas **web estáticas** tienen también **algunas ventajas**:

- **No es necesario saber programar** para crear un sitio que utilice únicamente páginas web estáticas. Simplemente habría que conocer HTML/XHTML y CSS, e incluso esto no sería indispensable: se podría utilizar algún programa de diseño web para generarlas.
- La característica diferenciadora de las páginas web estáticas es que **su contenido nunca varía**, y esto en algunos casos también **puede suponer una ventaja**. Sucede, por ejemplo, cuando quieres almacenar un enlace a un contenido concreto del sitio web: si la página es dinámica, al volver a visitarla utilizando el enlace su contenido puede variar con respecto a cómo estaba con anterioridad. O cuando quieres dar de alta un sitio que has creado en un motor de búsqueda como Google.
- Para que Google muestre un sitio en sus resultados, primero debe rastrear e indexar su contenido: un programa (Googlebot) recorre las páginas, consulta su contenido y lo guarda en su índice. Tanto las páginas estáticas como las dinámicas pueden indexarse, pero las dinámicas suponen más trabajo: si dependen de JavaScript o de la interacción del usuario, Googlebot puede tardar más en procesarlas o incluso no indexarlas del todo. Las estáticas, en cambio, son más fáciles de rastrear porque su HTML ya contiene todo el contenido desde el principio.
- Otra desventaja de las páginas dinámicas es que requieren que el servidor ejecute su código mediante un módulo concreto, integrado en el propio servidor (como `mod_php` en Apache) o como proceso independiente al que este delega la ejecución. Esto implica recursos adicionales que las páginas estáticas no necesitan.

Además, puede ser necesario consultar una base de datos como parte de la ejecución del programa. Es decir, la ejecución de una página web dinámica requiere una serie de recursos del lado del servidor.

Estos recursos deben instalarse y mantenerse. **Las páginas web estáticas sólo necesitan un servidor web que se comuniquen con tu navegador** para enviártela. Y de hecho para ver una página estática almacenada en tu equipo no necesitas siquiera de un servidor web. Son archivos que pueden almacenarse en un soporte de almacenamiento como puede ser un disco óptico o una memoria USB y abrirse desde él directamente con un navegador web.

Pero si decides hacer un sitio web utilizando **páginas estáticas**, ten en cuenta que **tienen limitaciones**. La desventaja más importante ya la comentamos anteriormente: la **actualización** de su contenido debe hacerse de **forma manual** editando la página que almacena el servidor web. Esto implica un mantenimiento que puede ser prohibitivo en sitios web con gran cantidad de contenido.

Las **primeras páginas web** que se crearon en Internet fueron páginas **estáticas**. A esta web compuesta por páginas estáticas se le considera la primera generación. La segunda generación de la web surgió gracias a las páginas web dinámicas. Tomando como base las web dinámicas, han ido surgiendo otras tecnologías que han hecho evolucionar Internet hasta llegar a lo que ahora conocemos.

1.2. APLICACIONES WEB.

Las **aplicaciones web** emplean **páginas web dinámicas** que se **ejecutan en un servidor web** y se muestran en un navegador. Puedes encontrar aplicaciones web para realizar múltiples tareas. Unas de las primeras en aparecer fueron las que viste antes, los clientes de correo, que te permiten consultar los mensajes recibidos y enviar correos directamente desde el navegador.

Hoy en día existen aplicaciones web para multitud de tareas como procesadores de texto, gestión de tareas, o edición y almacenamiento de imágenes. Estas aplicaciones tienen ciertas ventajas e inconvenientes si las comparas con las aplicaciones tradicionales que se ejecutan sobre el sistema operativo de la propia máquina.

Ventajas de las aplicaciones web:

- **No es necesario instalarlas en aquellos equipos en que se vayan a utilizar.** Se instalan y se ejecutan solamente en un equipo, en el servidor, y esto es suficiente para que se puedan utilizar de forma simultánea desde muchos equipos.
- Como solo se encuentran instaladas en un equipo, **es muy sencillo gestionarlas** (hacer copias de seguridad de sus datos, corregir errores, actualizarlas).
- **Se pueden utilizar en todos aquellos sistemas que dispongan de un navegador web**, independientemente de sus características (no es necesario un equipo potente) o de su sistema operativo.
- **Se pueden utilizar desde cualquier lugar en el que dispongamos de conexión con el servidor.** En muchos casos esto hace posible que se pueda acceder a las

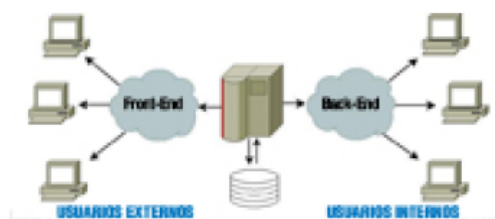
aplicaciones desde sistemas como los teléfonos móviles.

Inconvenientes de las aplicaciones web:

- **La interfaz de usuario de las aplicaciones web es la página que se muestra en el navegador**, lo que limita sus funcionalidades a lo que el navegador puede ofrecer.
- **Dependemos de una conexión con el servidor para poder utilizarlas**. Si nos falla la conexión, no podremos acceder a la aplicación web, a no ser que se trate de **aplicaciones web progresivas (PWA)** que permiten cierto funcionamiento offline.
- **La información que se muestra en el navegador debe transmitirse desde el servidor**. Esto hace que cierto tipo de aplicaciones no sean adecuadas para su implementación como aquellas que requieren acceso directo y de bajo nivel al hardware (por ejemplo, software de diseño 3D con aceleración gráfica específica o videojuegos muy exigentes que necesitan aprovechar al máximo la GPU local) no son adecuadas para su implementación como aplicación web.

Hoy en día muchas aplicaciones web utilizan las ventajas que les ofrece la generación de páginas dinámicas. La gran mayoría de su contenido está almacenado en una **base de datos**. Aplicaciones como **Drupal**, **Joomla!**, **WordPress** y otras muchas ofrecen dos partes bien diferenciadas:

- **Una parte externa o front-end**, que es el conjunto de páginas que ven la gran mayoría de **usuarios** que las usan (usuarios externos).
- **Una parte interna o back-end**, que es otro conjunto de páginas dinámicas que utilizan las personas que **producen el contenido** y las que **administran** la aplicación web (usuarios internos) para crear contenido, organizarlo, decidir la apariencia externa, etc.



1.2. EJECUCIÓN DE CÓDIGO EN EL SERVIDOR Y EN EL CLIENTE.

Como vimos, cuando tu navegador solicita una página a un servidor web, es posible que antes de enviarla haya tenido que ejecutar algún programa para obtenerla. Ese programa es el que genera, en parte o en su totalidad, la página web que llega a tu equipo. En estos casos, **el código se ejecuta en el entorno del servidor web**.

Además, cuando una página web llega a tu navegador, es también posible que

incluya algún programa o fragmentos de código que se deban ejecutar. Ese código, normalmente en lenguaje **JavaScript**, **se ejecutará en tu navegador** y, además de poder modificar el contenido de la página, también puede llevar a cabo acciones como la animación de textos u objetos de la página o la comprobación de los datos que introduces en un formulario.

Estas dos tecnologías se complementan entre sí. Así, volviendo al ejemplo del correo web, el programa que se encarga de obtener tus mensajes y su contenido de una base de datos se ejecuta en el entorno del servidor, mientras que tu navegador ejecuta, por ejemplo, el código encargado de avisar cuando quieres enviar un mensaje y te has olvidado de poner un texto en el asunto.

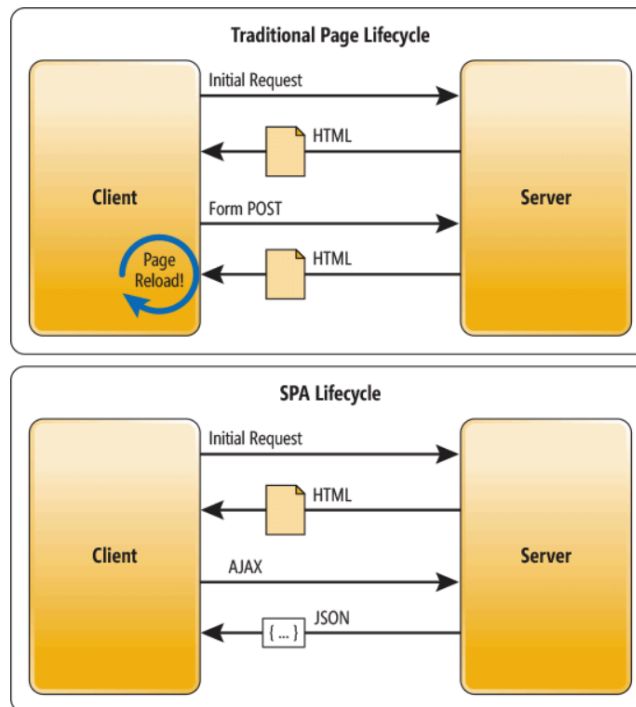
Esta división es así porque el **código que se ejecuta en el cliente** web (en tu navegador) no tiene, o mejor dicho **tradicionalmente no tenía, acceso a los datos** que se almacenan en el **servidor**. Es decir, cuando en tu navegador querías leer un nuevo correo, el código JavaScript que se ejecutaba en el mismo no podía obtener de la base de datos el contenido de ese mensaje. La solución era crear una nueva página en el servidor con la información que se pedía y enviarla de nuevo al navegador.

Sin embargo, desde hace unos años existe una **técnica de desarrollo web conocida como AJAX**, que nos posibilita realizar programas en los que el código JavaScript que se ejecuta en el navegador pueda comunicarse con un servidor de Internet para obtener información con la que, por ejemplo, modificar la página web actual.

La **arquitectura SPA** (Single Page Application) es un enfoque moderno en el desarrollo web que permite construir aplicaciones que funcionan dentro de una sola página HTML, ofreciendo una experiencia de usuario rápida, fluida y dinámica. En esta arquitectura la aplicación se carga una sola vez en el navegador. A partir de ahí, **no se recarga la página completa**: solo se actualizan partes específicas mediante JavaScript, utilizando tecnologías como AJAX, para comunicarse con el servidor y obtener datos sin recargar.

En nuestro ejemplo, cuando pulsas con el ratón encima de un correo que quieres leer, la página puede contener código JavaScript que detecte la acción y, en ese instante, consultar a través de Internet el texto que contiene ese mismo correo y mostrarlo en la misma página, modificando su estructura en caso de que sea necesario. Es decir, sin salir de una página se puede modificar su contenido en base a la información que se almacena en un servidor de Internet.

A día de hoy, gran parte del desarrollo web está pasando de una arquitectura web cliente-servidor clásica, donde el cliente realiza una llamada al backend, hacia una arquitectura SPA donde el cliente gana mucho mayor peso y sigue una programación reactiva que accede a servicios remotos REST que realizan las operaciones (comunicándose mediante JSON).



2. TECNOLOGÍAS PARA PROGRAMACIÓN WEB DEL LADO DEL SERVIDOR.

Cuando programas una aplicación, utilizas un lenguaje de programación. Por ejemplo, utilizas el lenguaje Java para crear aplicaciones que se ejecuten en distintos sistemas operativos. Al programar cada aplicación utilizas ciertas herramientas como un entorno de desarrollo o librerías de código. Además, una vez acabado su desarrollo, esa aplicación necesitará ciertos componentes para su ejecución, como por ejemplo una máquina virtual de Java.

En este bloque **vas a conocer las distintas tecnologías** que se pueden utilizar para **programar aplicaciones** que se ejecuten en un **servidor web**, y cómo se relacionan unas con otras. Verás las ventajas e inconvenientes de utilizar cada una, y qué lenguajes de programación deberás aprender para utilizarlas.

Los **componentes principales** con los que debes contar para ejecutar aplicaciones web en un servidor son los siguientes:

- Un **servidor web** para recibir las peticiones de los clientes web (normalmente navegadores) y enviarles la página que solicitan (una vez generada puesto que hablamos de páginas web dinámicas). El servidor web debe conocer el procedimiento a seguir para generar la página web: qué módulo se encargará de la ejecución del código y cómo se debe comunicar con él.
- El **módulo encargado de ejecutar el código** o programa y generar la página web resultante. Este módulo debe integrarse de alguna forma con el servidor web, y

dependerá del lenguaje y tecnología que utilicemos para programar la aplicación web.

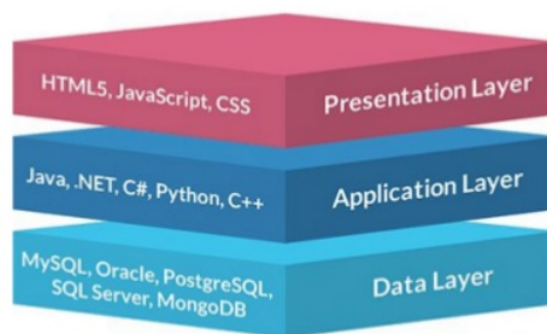
- Una **base de datos**, que normalmente también será un servidor. Este componente no es estrictamente necesario, pero en la práctica se utiliza en todas las aplicaciones web que utilizan grandes cantidades de datos para almacenarlos.
- El **lenguaje de programación** que utilizarás para desarrollar las aplicaciones (en nuestro caso, PHP).

Además de los componentes a utilizar, también es importante decidir cómo vas a **organizar el código** de la aplicación. Muchas de las arquitecturas que se usan en la programación de aplicaciones web te ayudan a estructurar el código de las aplicaciones en **capas o niveles**.

El motivo de dividir en capas el diseño de una aplicación es que se puedan **separar las funciones lógicas** de la misma, de tal forma que sea posible ejecutar cada una en un servidor distinto (en caso de que sea necesario).

Cada capa puede ocuparse de una o varias de las funciones anteriores. Por ejemplo, en las **arquitecturas de 3 capas** nos podemos encontrar con:

- Una **capa cliente (presentación)**, donde programamos todo lo relacionado con la interfaz de usuario, esto es, la parte visible de la aplicación con la que interactúa el usuario (HTML, CSS, JS).
- Una **capa de aplicación (lógica de negocio)** donde deberás programar la funcionalidad de tu aplicación (PHP, Java, Python, .NET, etc.)
- Una **capa de datos**, que se tendrá que encargar de almacenar la información de la aplicación en una base de datos y recuperarla cuando sea necesario.

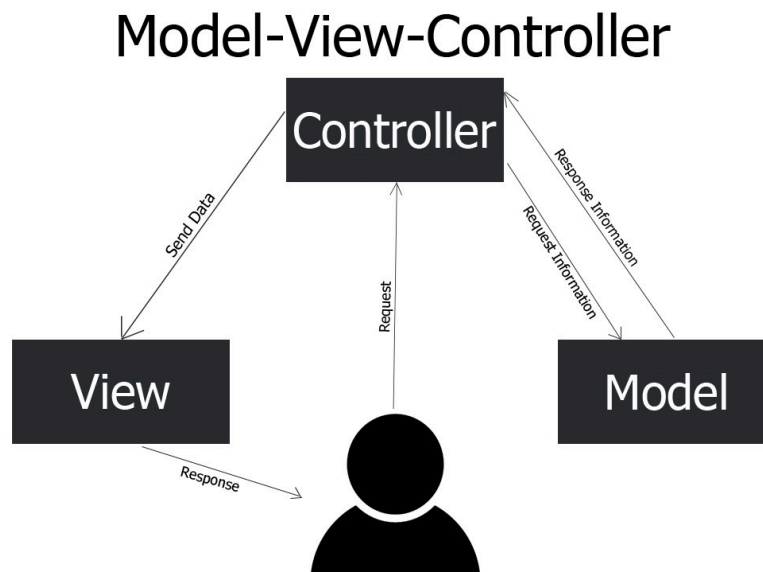


MODELO VISTA-CONTROLADOR (MVC)

El Modelo Vista Controlador (Model-View-Controller) es un modelo de arquitectura que separa los datos y la lógica de negocio respecto a la interfaz de usuario y el componente encargado de gestionar los eventos y las comunicaciones.

Al separar los componentes en elementos conceptuales permite reutilizar el código y mejorar su organización y mantenimiento. Sus elementos son:

- **Modelo:** datos y lógica de negocio. Representa la información y gestiona todos los accesos a ésta, tanto consultas como actualizaciones provenientes, normalmente, de una base de datos. Se accede vía el controlador.
- **Controlador:** intermediario modelo/vista. Responde a las acciones del usuario, y realiza peticiones al modelo para solicitar información. Tras recibir la respuesta del modelo, le envía los datos a la vista.
- **Vista:** interfaz de usuario y cómo se muestran los datos. Presenta al usuario de forma visual el modelo y los datos preparados por el controlador. El usuario interactúa con la vista y realiza nuevas peticiones al controlador.



Se estudia con más detalle al profundizar en el uso de los frameworks PHP.

2.1. ARQUITECTURAS Y PLATAFORMAS.

La primera elección que harás antes de comenzar a programar una aplicación web es la arquitectura o plataforma que vas a utilizar. Hoy en día, puedes elegir entre:

- **Jakarta EE:** antes Java EE (Enterprise Edition) y originalmente J2EE. Es una plataforma que reúne un conjunto de especificaciones, APIs y tecnologías para el desarrollo de aplicaciones empresariales en **Java**. Proporciona un conjunto de especificaciones y librerías para crear aplicaciones modulares, escalables y seguras, que pueden funcionar con distintos gestores de bases de datos y servidores de aplicaciones.

Está impulsada por la Fundación Eclipse y cuenta con el respaldo de empresas como Oracle, IBM o Red Hat. Una de sus grandes ventajas es la enorme cantidad de librerías y herramientas disponibles en Java, además de una amplia comunidad de desarrolladores.

Entre sus tecnologías más conocidas están Servlets y JSP (orientadas a la

generación dinámica de páginas web) y EJB (Enterprise JavaBeans), que encapsulan la lógica de negocio de las aplicaciones.

- **AMP** son las siglas de Apache, MySQL/MariaDB y PHP/Perl/Python. Las dos primeras siglas hacen referencia al servidor web (Apache) y al servidor de base de datos (MySQL o MariaDB). La última se corresponde con el lenguaje de programación utilizado, que puede ser PHP, Perl o Python, siendo PHP el más empleado de los tres.

Dependiendo del sistema operativo que se utilice para el servidor, se utilizan las siglas LAMP (para Linux), WAMP (para Windows) o MAMP (para Mac). También es posible usar otros componentes, como el gestor de bases de datos PostgreSQL en lugar de MySQL.

Todos los componentes de esta arquitectura son de código libre (open source). Es una plataforma de programación que permite desarrollar aplicaciones de tamaño **pequeño o mediano** con un aprendizaje sencillo. Su gran ventaja es la gran comunidad que la soporta y la multitud de aplicaciones de código libre disponibles.

Existen paquetes que agrupan todos los elementos de la pila en una sola instalación para facilitar el despliegue, como **XAMPP**, que está disponible para Windows, Linux y Mac y permite disponer rápidamente de un entorno completo de desarrollo.



¿Qué incluye XAMPP?

X → multiplataforma (funciona en Windows, Linux y Mac).

A → Apache (servidor web que atiende las peticiones).

M → MySQL / MariaDB (sistema gestor de base de datos).

P → PHP (lenguaje de programación de servidor más utilizado)

P → Perl (lenguajes de programación menos usado hoy en día).

Descargar desde: <https://www.apachefriends.org/>

- **CGI/Perl. CGI (Common Gateway Interface)** es un estándar que permite a un servidor web comunicarse con programas externos para generar contenido dinámico. Un programa CGI puede estar escrito en diferentes lenguajes, como **C, C++, Perl, Python o PHP**, entre otros.

CGI tuvo una gran importancia en los primeros años del desarrollo web dinámico. Sin embargo, su modelo tradicional presenta un problema de rendimiento: cada petición puede implicar la creación de un nuevo proceso para ejecutar el programa CGI, lo que supone un consumo elevado de recursos cuando existe un número importante de peticiones simultáneas. Por este motivo, en las aplicaciones web modernas se utilizan habitualmente otros mecanismos y servidores de aplicaciones más eficientes.

Perl tuvo un papel especialmente importante en los comienzos de la programación web mediante CGI, pero actualmente tiene una presencia mucho menor en el desarrollo web que tecnologías como PHP, Java, C# o JavaScript/Node.js.

- **ASP.NET Core** es el framework web de Microsoft para el desarrollo de aplicaciones y servicios web. Forma parte del ecosistema **.NET** y permite crear aplicaciones web dinámicas y APIs. Es la evolución de la tecnología **ASP.NET** y utiliza principalmente el lenguaje de programación **C#**.

A diferencia de las versiones antiguas de ASP.NET, **ASP.NET Core es multiplataforma**, por lo que las aplicaciones pueden ejecutarse en Windows, Linux o macOS. En Windows puede utilizarse **IIS (Internet Information Services)** como servidor web.

ASP.NET Core puede trabajar con diferentes sistemas gestores de bases de datos, como **SQL Server, MySQL o PostgreSQL**, y cuenta con un amplio ecosistema de librerías y herramientas de desarrollo.

Una de sus principales ventajas es que **.NET y ASP.NET Core son de código abierto y multiplataforma**, y disponen de herramientas de desarrollo como **Visual Studio**.

- **Node.js** es un entorno de ejecución de **código abierto** que permite ejecutar programas escritos en **JavaScript en el lado servidor**, es decir, fuera del navegador.

Permite desarrollar **aplicaciones web y servicios**, como APIs, y cuenta con un amplio ecosistema de módulos y herramientas que facilitan el desarrollo. Entre los frameworks más conocidos se encuentran **Express.js** y **NestJS**.

Una de sus principales ventajas es que permite desarrollar aplicaciones del lado servidor utilizando JavaScript.

2.1.1. SELECCIÓN DE UNA ARQUITECTURA DE PROGRAMACIÓN WEB.

Como has visto, hay muchas decisiones que debes tomar antes de comenzar el desarrollo de una aplicación web. La arquitectura que utilizarás, el lenguaje de programación, el entorno de desarrollo, el gestor de bases de datos, el servidor web, incluso cómo estructurar tu aplicación.

Antes de comenzar el desarrollo de una aplicación deberás considerar, entre otros, los siguientes aspectos:

- ¿Qué tamaño y complejidad tiene el proyecto?
- ¿Qué lenguaje o lenguajes de programación conozco?
- ¿Qué tecnologías son las más adecuadas para las características de la aplicación?
- ¿Es necesario aprender una nueva tecnología para desarrollar el proyecto?
- ¿Qué frameworks, librerías y herramientas están disponibles?
- ¿Voy a utilizar software de código abierto, software comercial o una combinación de ambos?
- ¿Qué costes pueden tener las herramientas, servidores, servicios y licencias que voy a utilizar?
- ¿Voy a desarrollar la aplicación individualmente o formo parte de un equipo?
- ¿Qué servidor web y qué sistema gestor de bases de datos voy a utilizar?
- ¿Dónde se desplegará la aplicación: servidor propio, máquina virtual, contenedor, nube, etc.?
- ¿Qué requisitos de seguridad, rendimiento, escalabilidad y mantenimiento tiene el proyecto?
- ¿Qué conocimientos y experiencia tiene el equipo de desarrollo?
- ¿Qué licencia se aplicará al software desarrollado?

Analizando las respuestas a estas preguntas podremos determinar qué **arquitectura, lenguaje, framework y herramientas** se adaptan mejor a las características de nuestra aplicación.

Autoevaluación

¿Cuál de estas tecnologías permite la ejecución por el servidor web de programas escritos en cualquier lenguaje?

Java EE

PHP

AMP

CGI

2.2. INTEGRACIÓN CON EL SERVIDOR WEB.

La comunicación entre un cliente web (navegador) y un servidor web se realiza mediante el protocolo HTTP, que actúa como vínculo entre el usuario y la aplicación web. Cada acción que realiza el usuario —como enviar un formulario— se transmite como una petición HTTP, y la respuesta del servidor llega de vuelta como una respuesta HTTP.

En el lado del servidor, estas peticiones son recibidas y gestionadas por el servidor web (o servidor HTTP), que decide cómo procesarlas y, si es necesario, delegar en otros componentes la ejecución del código de la aplicación. Cada tecnología web se integra con el servidor de forma diferente.

CGI (Common Gateway Interface) fue la primera forma estandarizada de generar contenido dinámico. Está disponible en casi todas las plataformas y define cómo un servidor web puede invocar un programa externo para que genere una página. Estos programas se llaman scripts CGI, sin importar el lenguaje en que estén escritos (aunque tradicionalmente se usó mucho Perl).

El principal inconveniente de CGI es que para cada petición se crea un nuevo proceso, lo que consume más recursos y ralentiza las respuestas. Para mejorar el rendimiento surgieron soluciones como FastCGI y módulos específicos que permiten ejecutar el código dentro del propio servidor web sin crear procesos nuevos. Por ejemplo, `mod_perl` para Perl en Apache.

Con PHP ocurre algo similar: aunque podría ejecutarse como CGI, lo más habitual en entornos tipo AMP es usar el módulo ***mod_php*** (o en versiones modernas, PHP-FPM con FastCGI), que es más eficiente. Para Python existía un enfoque parecido llamado `mod_python`, aunque el proyecto está descontinuado desde hace años; hoy se usa **WSGI (Web Server Gateway Interface)**, un estándar que define cómo debe comunicarse un servidor web con una aplicación Python, normalmente a través de un servidor de aplicaciones situado detrás de un servidor web (Apache o Nginx) que actúa como proxy inverso. .

En el caso de Jakarta EE (antes Java EE y originalmente J2EE), la arquitectura es más compleja. Para ejecutar aplicaciones Jakarta EE podemos usar:

- Servidores de aplicaciones completos, que implementan todas las especificaciones de la plataforma.
- Contenedores de servlets, que solo soportan parte de la especificación y resultan más ligeros.

La elección depende del tamaño y las tecnologías que requiera la aplicación. Entre los servidores de aplicaciones Java EE más conocidos se encuentran las soluciones comerciales IBM WebSphere y ORACLE WebLogic, y las de código abierto como

JBoss/WildFly, GlassFish o Apache Geronimo (este último lleva años inactivo).

3. LENGUAJES DE SERVIDOR.

Los lenguajes de programación web se diferencian, entre otras cosas, por cómo se ejecutan en el servidor. Hay tres tipos:

- **Lenguajes de guiones o scripting:** estos lenguajes se ejecutan directamente desde su código fuente a través de un intérprete, que procesa las instrucciones y genera la página web. Normalmente se almacenan en archivos de texto plano. Ejemplos típicos son PHP, Python, Perl y ASP clásico. Su principal ventaja es la portabilidad y la posibilidad de modificar el código y ver los cambios de inmediato, pero su rendimiento suele ser inferior al de los lenguajes compilados, ya que cada petición se interpreta de nuevo.
- **Lenguajes compilados a código máquina:** en este caso, el código fuente se traduce directamente a código ejecutable. El servidor ejecuta directamente este código, normalmente a través de CGI, lo que permite usar lenguajes de propósito general como C. Son muy rápidos, pero tienen dos inconvenientes: por un lado, poca portabilidad, ya que un binario compilado para una plataforma concreta no funciona en otra sin recompilar; por otro, poca integración con el servidor web, ya que normalmente cada petición genera un nuevo proceso (típicamente vía CGI), lo que aumenta el consumo de recursos. .
- **Lenguajes compilados a código intermedio:** aquí, el código fuente se convierte a un código intermedio independiente del procesador, que luego se ejecuta en una máquina virtual, la cual puede además compilar en caliente (JIT, Just-In-Time) las partes más usadas del código a código máquina para mejorar el rendimiento. Esto ocurre, por ejemplo, en aplicaciones Java (servlets, JSP, Jakarta EE) y ASP.NET. Ofrecen un buen equilibrio entre rendimiento y portabilidad, permiten reutilizar procesos y recursos del servidor, aunque requieren una plataforma específica.

En resumen, los lenguajes scripting destacan por su simplicidad y portabilidad, los compilados a máquina por su velocidad, y los compilados a código intermedio combinan rendimiento con compatibilidad entre distintas plataformas, siendo ideales para aplicaciones web de mayor tamaño y complejidad.

3.1. CÓDIGO EMBEBIDO EN EL LENGUAJE DE MARCAS.

Una de las principales formas de realizar páginas web dinámicas es **integrar el**

código del programa en medio de las etiquetas HTML de la página web. De esta forma, el contenido que no varía de la página se puede introducir directamente en HTML, y el lenguaje de programación se utilizará para todo aquello que pueda variar de forma dinámica.

Por ejemplo, puedes incluir dentro de una página HTML un pequeño código en lenguaje PHP que muestre el nombre del servidor:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Nombre del Servidor</title>
</head>
<body>
  <h1>Información del Servidor</h1>
  <p>Este sitio está siendo servido desde:
    <?php
      // Mostramos el nombre del servidor
      echo $_SERVER['SERVER_NAME'];
    ?>
  </p>
</body>
</html>
```

Esta metodología de programación es la que se emplea en los lenguajes **ASP, PHP y en páginas JSP de Jakarta EE.**

3.2. HERRAMIENTAS DE PROGRAMACIÓN.

Dos de los IDE de código abierto más utilizados en la actualidad son Eclipse y NetBeans. Ambos permiten el desarrollo de aplicaciones informáticas en varios lenguajes de programación. Aunque en sus orígenes se centraron en la programación en lenguaje Java, hoy en día admiten directamente o a través de módulos, varios lenguajes entre los que se incluyen C, C++, PHP, Python y Ruby.

Ambos ofrecen para la descarga **versiones personalizadas del IDE, que pueden ser usadas directamente para programar en un lenguaje determinado**, sin necesidad de cambiar la configuración o instalar módulos.

En este curso vamos a emplear **Visual Studio Code** (<https://code.visualstudio.com>) como entorno de desarrollo (IDE). Existen otras alternativas, siendo **PhpStorm** la más conocida pero de pago. Otra posibilidad es utilizar **Eclipse**, aunque es un entorno bastante pesado.

VSCoide es un editor de código fuente que se complementa mediante extensiones.

Para facilitar el trabajo a lo largo del curso vamos a utilizar las siguientes extensiones:

- **PHP Intelephense**: autocompletado inteligente (sugiere funciones, clases, métodos y variables mientras escribes), información contextual (al pasar el cursor sobre una función o clase, muestra documentación oficial o comentarios de PHP_Doc), formato de código (aplica estilos como PSR-12 para mantener tu código limpio y consistente), diagnóstico de errores en tiempo real, soporte para HTML/JS/CSS embebido y también permite saltar rápidamente a la definición de una función, clase o método y ver dónde se usa en todo el proyecto.
- **PHP Code Sniffer**: detecta errores de estilo y estructura en tu código.
- **Code Runner**: hay que configurarlo para que ejecute el código en la terminal integrada y funcione con PHP puro, sin frameworks.
- **Laravel Snippets**.

3.3. PROGRAMACIÓN WEB CON PHP.

PHP es un lenguaje interpretado de propósito general diseñado para el desarrollo de páginas web dinámicas mediante la inserción de código embebido dentro del lenguaje de marcas HTML. Su sintaxis está basada en la de C/C++, y por lo tanto es muy similar a la de Java.

Aunque se pueden usar delimitadores abreviados o `short_open_tag`, los **delimitadores** recomendados para incluir código PHP dentro de una página web son `<?php` y `?>`. En archivos sólo PHP puros no se incluye el delimitador de cierre.

Actualmente la versión recomendada es PHP 8.x (siempre superior a la 7.0). PHP dispone de una multitud de librerías y frameworks (Laravel, Symfony, Codeigniter, Zend).

El código PHP es ejecutado por un entorno de ejecución con el que se integra el servidor web (normalmente utilizando Apache con el módulo **mod_php**). La configuración tanto del servidor web Apache, como de PHP, se realiza por medio de ficheros de configuración. El de Apache es **httpd.conf** y el de PHP es **php.ini**. Este fichero, **php.ini**, puede encontrarse en distintas ubicaciones. La función **phpinfo()** que ejecutaste antes te informa, entre otras muchas cosas, del lugar en que se encuentra almacenado el fichero **php.ini** en tu ordenador. Si utilizamos xampp sería **C:\xampp\php\php.ini**.

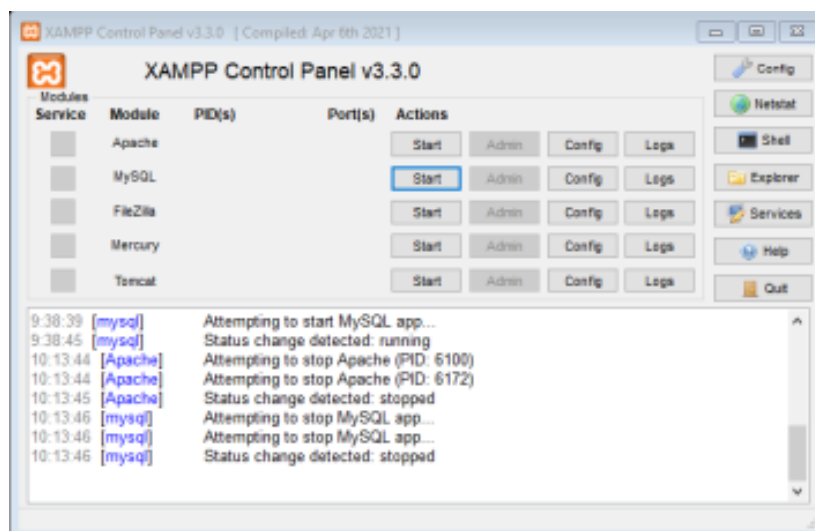
Algunas de las directivas más utilizadas que figuran en el fichero **php.ini** son:

- **short_open_tag**. Indica si se pueden utilizar en PHP los delimitadores cortos `<?` y `?>`. Es preferible no usarlos, pues puede causarnos problemas si utilizamos páginas con XML. Para prohibir la utilización de estos delimitadores con PHP le asignamos a esta directiva el valor Off.

- **max_execution_time**. Permite que puedas ajustar el número máximo de segundos que podrá durar la ejecución de un script PHP. Evita que el servidor se bloquee si se produce algún error en un script.
- **error_reporting**. Indica qué tipo de errores se mostrarán en el caso de que se produzcan. Por ejemplo, si haces **error_reporting = E_ALL**, te mostrará todos los tipos de errores. Si no quieres que te muestre los avisos pero sí otros tipos de errores, puedes hacer **error_reporting = E_ALL & ~E_NOTICE**.
- **file_uploads**. Indica si se pueden o no subir ficheros al servidor por HTTP.
- **upload_max_filesize**. En caso de que se puedan subir ficheros por HTTP, puedes indicar el límite máximo permitido para el tamaño de cada archivo. Por ejemplo, **upload_max_filesize = 1M**.

4. PUESTA EN MARCHA.

Comenzamos con el típico programa en PHP que muestre “Hola mundo” en el navegador. Para ello arrancamos XAMPP e iniciamos Apache en el panel de control (start):



Para comprobar que también el servidor de la base de datos funciona correctamente, ejecutaremos desde el navegador la aplicación **phpMyAdmin** que nos proporciona el paquete y que permite la administración de la base de datos.

La carpeta **htdocs** es la conocida como **DocumentRoot**, en la que Apache buscará todas las aplicaciones PHP. Por tanto, para ejecutarlas, deben estar guardadas en principio en esa carpeta. El contenido inicial de la misma es este:

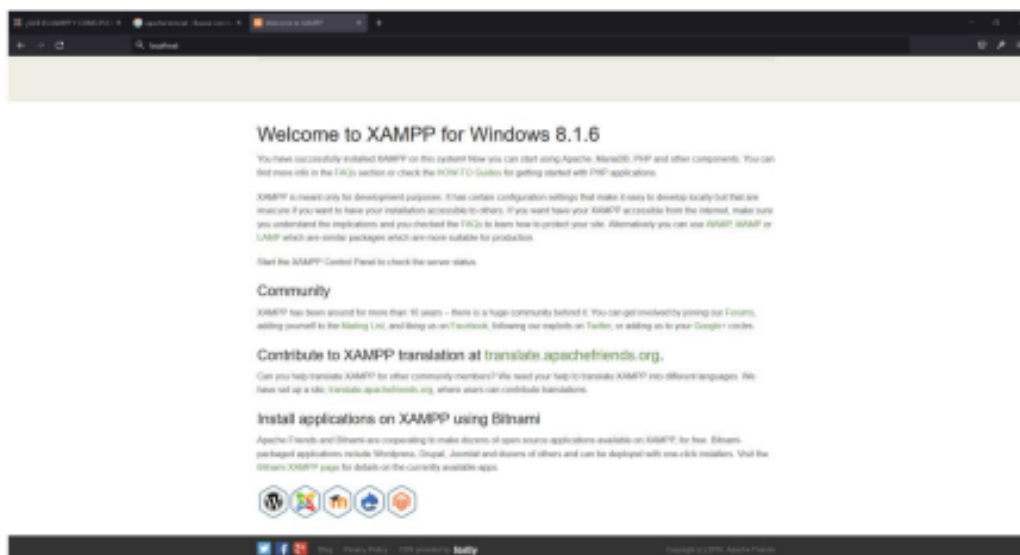
Este equipo > Disco local (C:) > xampp > htdocs >			
Ordenar Ver ...			
Nombre	Fecha de modificación	Tipo	Tamaño
dashboard	11/03/2025 18:16	Carpeta de archivos	
img	02/09/2025 17:32	Carpeta de archivos	
webalizer	11/03/2025 18:16	Carpeta de archivos	
xampp	11/03/2025 18:16	Carpeta de archivos	
applications	15/06/2022 18:07	Firefox HTML Doc...	4 KB
bitnami	15/06/2022 18:07	Archivo de origen ...	1 KB
favicon	16/07/2015 17:32	Archivo ICO	31 KB
index	16/07/2015 17:32	Archivo de origen ...	1 KB

En la imagen anterior se ve un archivo **index.php**. Apache está configurado para que, al acceder a un directorio, cualquier archivo con el nombre de index.php se ejecute de forma automática, y por eso aparece la pantalla que vimos al principio, para comprobar que Apache funciona.

Si le cambiamos el nombre a este fichero o lo eliminamos, el navegador mostraría todos los archivos y directorios que haya dentro del DocumentRoot (a esto se le llama el listado de directorios en servidores web).

Ahora probaremos en Xampp desde el navegador, para lo que utilizaremos el nombre de **localhost**, ya que de esta manera se accede a servicios locales.

Si todo funciona correctamente, nos aparecerá la siguiente pantalla:



A continuación crearemos **holamundo.php** (en C:/xampp/htdocs):

```
<!DOCTYPE html>
<html lang="es">
```

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Hola Mundo</title>
</head>
<body>
  <?php
    echo "Hola Mundo";
  ?>
</body>
</html>
```